

Sveučilište Josipa Jurja Strossmayera u Osijeku

Fakultet primijenjene matematike i informatike

MINESWEEPER U JACKU

Implementacija igre i automatskog Solvera na Hack platformi

TEHNIČKA DOKUMENTACIJA

Autori:

Noa Rakovac

Petar Pavan

David Sorgho

Osijek, 2026.

Sadržaj

- 1 Uvod
 - 1.1 O projektu
 - 1.2 Cilj projekta
 - 1.3 Načini igre
- 2 Arhitektura projekta
 - 2.1 Korištene tehnologije
 - 2.2 Struktura direktorija
 - 2.3 Uloga pojedinih klasa
 - 2.4 Kako program radi od pokretanja
- 3 Korisničko sučelje
 - 3.1 Glavni izbornik
 - 3.2 Ploča i kontrole
 - 3.3 Custom način
- 4 Board i stanje igre
 - 4.1 mines[] i state[]
 - 4.2 Otvaranje polja i prvi siguran potez
 - 4.3 Zastavica i upitnik
 - 4.4 Pobjeda i otvaranje praznih područja
- 5 Automatski Solver
 - 5.1 Što Solver smije vidjeti
 - 5.2 Obvious korak
 - 5.3 Constraint korak
 - 5.4 Guess i procjena rizika
- 6 Hint i djelomično ponovno crtanje
 - 6.1 Hint
 - 6.2 Partial redraw
- 7 Random, Maths i timer
- 8 Instalacija i pokretanje
- 9 Testiranje i ograničenja
- 10 Zaključak

1 Uvod

1.1 O projektu

U ovom projektu napravili smo klasičnu igru Minesweeper u programskom jeziku Jack za Hack platformu iz Nand2Tetris okruženja. Igra se može igrati ručno tipkovnicom, ali smo uz osnovnu igru dodali i automatski Solver koji pokušava riješiti ploču na temelju informacija koje su vidljive igraču.

Osim Solvera dodali smo Hint, zastavicu i upitnik, više razina težine, Custom ploču, Demo način, siguran prvi potez i djelomično ponovno crtanje ekrana. Kod smo podijelili u više klasa kako bi svaka imala jasnu ulogu i kako bi nam bilo lakše održavati projekt.

1.2 Cilj projekta

Glavni cilj bio nam je napraviti funkcionalan Minesweeper koji radi unutar ograničenja Hack računala, ali da projekt ne ostane samo na osnovnoj igri.

- omogućiti igranje preko tipkovnice na više razina težine
- napraviti odvojeni model stvarnih mina i stanja koje vidi igrač
- napraviti Solver koji ne čita stvarni raspored mina
- dodati Hint kao posebnu pomoć igraču
- osigurati da prvi potez bude siguran
- smanjiti nepotrebno ponovno crtanje cijele ploče
- omogućiti Custom veličinu ploče i broj mina

1.3 Načini igre

Način	Veličina	Broj mina	Tipka
Easy	9 × 9	10	1 / E
Medium	16 × 10	25	2 / M
Hard	20 × 10	44	3 / H
Demo	16 × 10	25, fiksno	D
Custom	3-20 × 3-10	ovisno o veličini	4 / C

2 Arhitektura projekta

2.1 Korištene tehnologije

Projekt je pisan u Jacku i pokreće se preko službenih Nand2Tetris alata. JackCompiler prevodi .jack datoteke u .vm datoteke, a VMEmulator ih zatim izvršava. Za crtanje i unos koristimo standardne Jack OS klase kao što su Screen, Output, Keyboard i Sys.

Dio	Tehnologija	Za što je koristimo
Izvorni kod	Jack	logika igre i objektna struktura
Prevođenje	JackCompiler	.jack → .vm
Pokretanje	VMEmulator	izvršavanje cijelog programa
Grafika	Screen / Output	ploča, tekst i HUD
Ulaz	Keyboard	tipke i kretanje
Vrijeme	Sys.wait	približni timer i pauze

2.2 Struktura direktorija

```
MineSweeper/  
├── Board/  
│   ├── Board.jack  
│   └── Board.vm
```

```

├─ Game/
│  ├─ Game.jack
│  └─ Game.vm
├─ Main/
│  ├─ Main.jack
│  └─ Main.vm
├─ Maths/
│  ├─ Maths.jack
│  └─ Maths.vm
├─ Random/
│  ├─ Random.jack
│  └─ Random.vm
├─ Solver/
│  ├─ Solver.jack
│  └─ Solver.vm
└─ README.md

```

Svaka klasa je u svojoj mapi. Ako kompajliramo svaku mapu posebno, prije pokretanja sve dobivene .vm datoteke stavimo u isti direktorij i taj direktorij učitamo u VMEmulator.

2.3 Uloga pojedinih klasa

Klasa	Uloga
Main	Ulazna točka programa. Inicijalizira Random, napravi Game i pokreće igru.
Game	Vodi meni, tipke, kursor, timer, restart, razine i pozive prema Boardu i Solveru.
Board	Čuva mine i stanje svakog polja te provodi pravila igre, Hint i crtanje.
Solver	Pokušava riješiti ploču pomoću vidljivih podataka iz Boarda.
Random	Generira pseudo-slučajne brojeve za raspored mina i odabire.
Maths	Sadrži pomoćnu funkciju mod za ostatak pri dijeljenju.

2.4 Kako program radi od pokretanja

Program kreće iz Main.main(). Prvo inicijaliziramo Random, zatim napravimo Game i pozovemo game.run(). Game prikazuje izbornik, stvara odgovarajući Board i Solver, a nakon toga u glavnoj petlji čita tipke i izvodi poteze.

```

do Random.init();
let game = Game.new();
do game.run();
do game.dispose();
do Sys.halt();

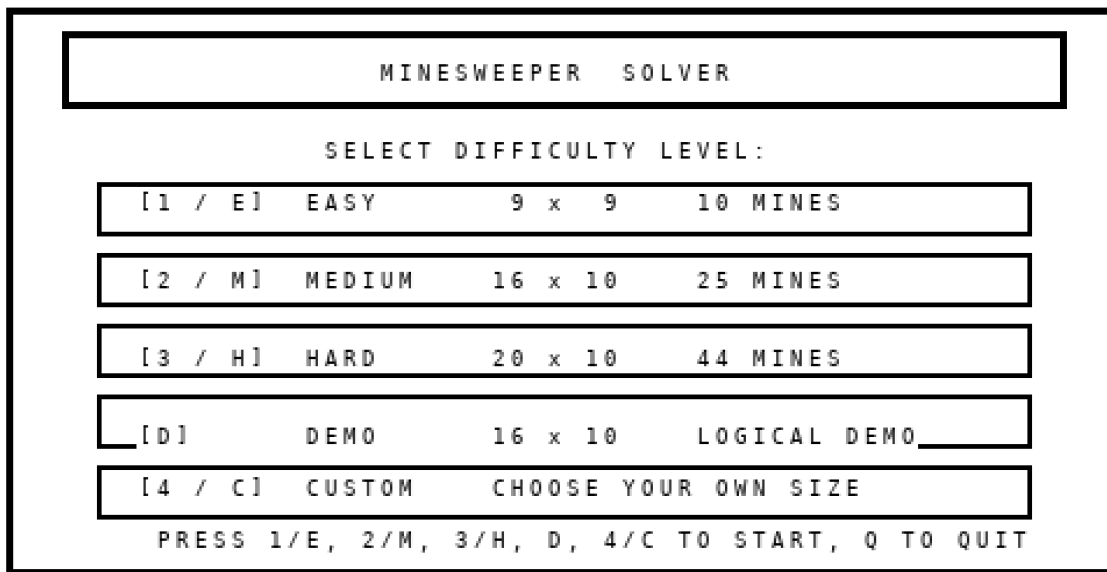
```

Game i Solver rade nad istim objektom Board. Zato kada Solver preko Boarda otvori ili označi polje, ta promjena je odmah dio stvarne igre i vidi se pri sljedećem crtanju.

3 Korisničko sučelje

3.1 Glavni izbornik

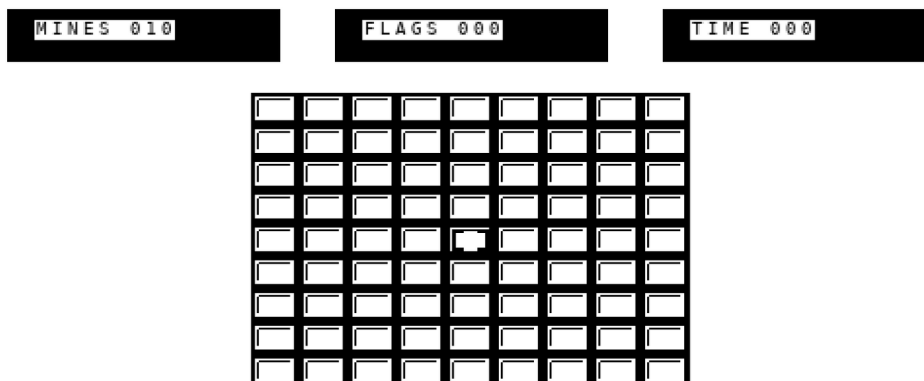
Na početku korisnik bira Easy, Medium, Hard, Demo ili Custom način. Htjeli smo da se cijela igra može koristiti samo tipkovnicom, bez miša, jer se to dobro uklapa u Nand2Tetris okruženje.



Slika 1: Glavni izbornik igre.

3.2 Ploča i kontrole

Na vrhu prikazujemo broj mina, broj postavljenih zastavica i vrijeme. Kursor se pomiče strelicama, Space otvara polje, F mijenja oznaku, H daje Hint, S radi jedan Solver korak, a A pokreće automatsko rješavanje.



Slika 2: Izgled igraće ploče na Easy razini.

Tipka	Akcija
Strelice	pomicanje kursora
Space	otvaranje polja
F	skriveno → zastavica → upitnik → skriveno
H	Hint - otvara jedno sigurno polje
S	jedan korak Solvera
A	automatsko rješavanje
R	restart
D	Demo
L	povratak na meni
Q	izlaz

3.3 Custom način

U Custom načinu korisnik bira širinu od 3 do 20, visinu od 3 do 10 i broj mina. Ograničenja smo stavili zato da ploča stane na Hack ekran i da ostane dovoljno sigurnih polja za normalan početak igre.

Broj mina mora biti između 1 i $\text{width} \times \text{height} - 10$. Time uvijek ostaje najmanje deset polja bez mina.

4 Board i stanje igre

4.1 mines[] i state[]

Jedna od važnijih stvari u projektu je da odvojeno čuvamo stvarni raspored mina i ono što korisnik vidi. Niz mines[] govori gdje stvarno postoje mine, dok state[] govori u kakvom je stanju određeno polje.

state	Stanje	Značenje
0	skriveno	polje nije otvoreno
1	otkriveno	prikazuje broj ili prazno polje
2	zastavica	igrač misli da je na polju mina
3	upitnik	igrač nije siguran; Solver ga tretira kao nepoznato polje

Ploča je logički dvodimenzionalna, ali Jack Array je jednodimenzionalan. Zato koordinate pretvaramo u indeks formulom:

$$\text{index} = x + y * \text{width}$$

Isti indeks koristimo za mines[], state[] i prevState[].

4.2 Otvaranje polja i prvi siguran potez

Metoda reveal(x,y) otvara polje i vraća true samo ako je stvarno pogodena mina. Ako je potez siguran, polje postaje otkriveno i po potrebi se proširi prazno područje.

Prvi potez smo napravili sigurnim. Na prvom valjanom otvaranju provjerimo kliknuto polje i njegovu 3×3 okolinu. Ako se u toj zoni nalazi mina, premjestimo je na drugo slobodno mjesto. Tako korisnik ne može izgubiti odmah na prvom potezu.

4.3 Zastavica i upitnik

Tipka F ciklički mijenja oznaku polja: skriveno → zastavica → upitnik → skriveno. Upitnik je samo vizualna napomena igraču. Za Solver je takvo polje i dalje nepoznato, isto kao i obično skriveno polje.



Slika 3: Primjeri oznaka koje se koriste na ploči.

4.4 Pobjeda i otvaranje praznih područja

Pobjeda se provjerava tako da više ne postoji nijedno sigurno polje koje je ostalo neotvoreno. Nije potrebno da sve mine budu označene zastavicama.

Kod otvaranja polja s nulom susjednih mina koristimo iterativno širenje. Više puta prolazimo pločom i otvaramo susjedna sigurna polja dok više nema novih promjena. Nismo koristili rekursiju jer nam je ovakav pristup jednostavniji i sigurniji za ograničeno Jack/Hack okruženje.

5 Automatski Solver

5.1 Što Solver smije vidjeti

Solver ne čita mines[] izravno. To nam je bilo bitno jer smo htjeli da rješava igru kao igrač, a ne da samo pročita gdje su mine.

Za informacije o ploči koristi metode kao što su `getVisibleState()`, `getVisibleNumber()` i `isHidden()`. `getVisibleState()` vraća stanje 0, 1, 2 ili 3, odnosno -1 ako su koordinate izvan ploče. `getVisibleNumber()` daje broj susjednih mina samo za otvoreno sigurno polje.

`isHidden()` vraća `true` i za skriveno polje i za upitnik. Tako Solver na jednom mjestu može pitati je li susjed još nepoznat, bez posebnog provjeravanja state 0 i state 3.

5.2 Obvious korak

Prvo pokušavamo pronaći potez iz osnovnih Minesweeper pravila. Za otvoreno brojčano polje Solver zna njegov broj, koliko je oko njega zastavica i koliko je još nepoznatih polja.

`remaining = number - flagged`

ako je `remaining = 0`:

preostala nepoznata polja su sigurna

ako je `remaining = unknown`:

sva nepoznata polja moraju biti mine

5.3 Constraint korak

Ako osnovna pravila nisu dovoljna, uspoređujemo skupove nepoznatih susjeda dvaju otvorenih brojeva. Ako je skup A sadržan u skupu B, iz razlike ta dva skupa možemo zaključiti što vrijedi za dodatna polja.

Primjer: $A = \{x, y\}$ ima jednu preostalu minu, a $B = \{x, y, z\}$ također jednu. Budući da je potrebna mina već unutar $\{x, y\}$, polje z mora biti sigurno. Ako bi B trebao imati dvije mine, tada bi z morao biti mina.

5.4 Guess i procjena rizika

Ako nema sigurnog logičkog poteza, Solver prelazi na guess. Za svako nepoznato polje procjenjuje rizik pomoću otvorenih brojeva u njegovoj okolini.

$\text{lokalni rizik} = (\text{preostale mine} \times 100) / \text{nepoznata susjedna polja}$

Ako polje dobiva informacije iz više otvorenih brojeva, uzimamo prosjek tih lokalnih procjena. Ako nema lokalnih informacija, koristimo globalni omjer preostalih mina i nepoznatih polja.

To nije savršena matematička vjerojatnost nego heuristika. Zbog toga Solver ne može garantirati pobjedu na svakoj mogućoj Minesweeper ploči.

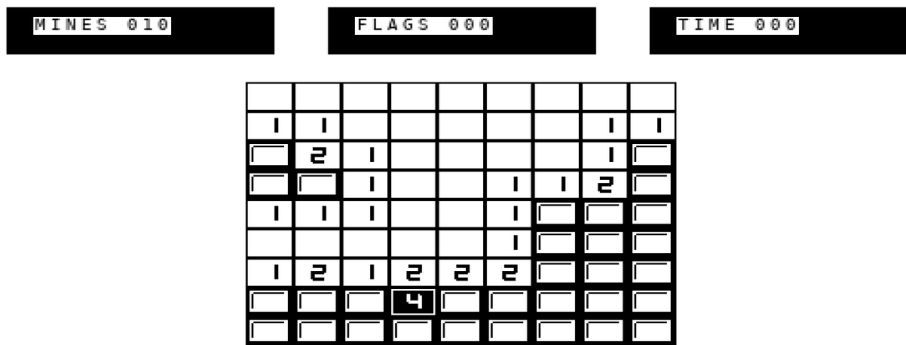
6 Hint i djelomično ponovno crtanje

6.1 Hint

Hint je odvojen od Solvera. Za razliku od Solvera, Hint smije provjeriti mines[] jer je njegova svrha samo pomoći igraču i garantirano otvoriti sigurno polje.

- `hasHiddenSafeNeighbor()` provjerava postoji li siguran skriven susjed
- `countHiddenSafeNeighbors()` broji koliko takvih kandidata postoji
- `getNthHiddenSafeNeighbor()` dohvaća odabranog kandidata po redu

- `giveHint()` bira kandidata, pamti njegove koordinate i otvara ga



Slika 4: Primjer otvorene ploče nakon korištenja pomoći.

6.2 Partial redraw

Hack platforma je dosta ograničena pa nam nije imalo smisla nakon svakog poteza ponovno crtati cijelu ploču. Zato Board pamti što je bilo nacrtano u prethodnom potezu.

`prevState[]` pamti prethodno stanje svakog polja, a dodatno pamtimo prethodni položaj kursora, prethodni Hint i `prevShowMines`. Kod sljedećeg `draw()` poziva ponovno crtamo samo polja kod kojih se nešto vizualno promijenilo.

Time smanjujemo broj poziva prema Screen funkcijama i izbjegavamo nepotrebno treperenje ekrana.

7 Random, Maths i timer

7.1 Random i Maths

Za postavljanje mina koristimo jednostavan pseudo-slučajni generator. Početni seed je 12345, a nova vrijednost računa se iz prethodne vrijednosti. Za prebacivanje broja u željeni interval koristimo vlastitu funkciju `mod` iz klase `Maths`.

```
seed = seed * 109 + 1021
```

```
mod(x, y) = x - y * (x / y)
```

Ovaj generator nije namijenjen kriptografiji, ali je sasvim dovoljan za raspored mina u igri.

7.2 Timer

Hack nema klasičan real-time sat koji bismo koristili kao u modernoj aplikaciji. Zato smo timer napravili približno pomoću `Sys.wait(50)`. Brojimo tickove i oko 20 tickova tretiramo kao jednu sekundu. Zbog brzine emulatora timer nije apsolutno precizan, ali je dovoljan za prikaz vremena partije.

8 Instalacija i pokretanje

8.1 Preuvjeti

- Nand2Tetris alati
- JackCompiler
- VMEulator

- projektne .jack ili već kompajlirane .vm datoteke

8.2 Pokretanje

Ako mijenjamo neku .jack datoteku, ponovno je kompajliramo JackCompilerom. Primjer na Windowsu:

```
cd C:\Users\<korisnik>\Downloads\nand2tetris\nand2tetris\tools
.\JackCompiler.bat "..\projects\MinesweeperFinal"
```

Nakon kompajliranja u zajedničkoj mapi trebaju biti Board.vm, Game.vm, Main.vm, Maths.vm, Random.vm i Solver.vm. U VMEmulatoru zatim učitamo cijelu mapu programa, a ne jednu pojedinačnu .vm datoteku.

9 Testiranje i ograničenja

9.1 Što smo provjeravali

- pokretanje Easy, Medium, Hard, Demo i Custom načina
- pomicanje kursora i otvaranje polja
- ciklus zastavica → upitnik → skriveno
- siguran prvi potez
- otvaranje praznih područja
- Hint i njegovo otvaranje sigurnog polja
- jedan Solver korak i Auto-solve
- pobjedu, poraz i restart
- djelomično ponovno crtanje promijenjenih polja

9.2 Ograničenja

Najveća ograničenja dolaze iz same Hack platforme. Ekran je 512×256 i grafika je jednostavna, pa smo morali ograničiti maksimalnu veličinu Custom ploče i paziti koliko često crtamo.

Solver također nije nepogrešiv. Kada obvious i constraint logika ne mogu dokazati siguran potez, mora napraviti procjenu rizika. U takvoj situaciji može pogoditi minu jer nije moguće iz vidljivih podataka uvijek zaključiti sigurno rješenje.

Demo način zato koristi fiksnu ploču koja nam služi za stabilnu demonstraciju logičkog ponašanja Solvera. I u Demo Solver i dalje koristi samo vidljive informacije s ploče.

10 Zaključak

Projekt nam je pokazao kako se i na dosta ograničenoj platformi može napraviti kompletna igra s više povezanih dijelova. Najviše posla bilo je oko Solvera, constraint logike, sigurnog prvog poteza i optimizacije crtanja.

Važan dio projekta je to što smo odvojili stvarni raspored mina od stanja koje vidi igrač. Zbog toga Solver ne dobiva rješenje unaprijed nego mora donositi odluke iz vidljivih brojeva, zastavica i nepoznatih polja.

Uz osnovni Minesweeper dobili smo i Hint, Custom način, Demo, automatsko rješavanje i partial redraw, pa projekt osim same igre pokazuje rad s algoritmima, objektima, nizovima, grafikom i ograničenjima Hack računala.

Literatura i korisni izvori

1. Nisan, N.; Schocken, S. The Elements of Computing Systems: Building a Modern Computer from First Principles, 2. izdanje, MIT Press, 2021.
2. Nand2Tetris - Project 9: High-Level Programming (Jack). <https://www.nand2tetris.org/project09>
3. Nand2Tetris - Software Suite i alati (JackCompiler, VM Emulator). <https://www.nand2tetris.org/software>
4. W3Schools - Java Arrays. https://www.w3schools.com/java/java_arrays.asp
5. Jason Bui - Nand2Tetris Part 2: Provided Jack Compiler Demonstration, YouTube, 2021. https://www.youtube.com/watch?v=_o-QJxng-X0
6. Sahil Vasava - Jack Programming Language | Computer from First Principles | Nand2Tetris 2 | Part 18, YouTube, 2021. <https://www.youtube.com/watch?v=6j6dybT0DCU>